

Data Engineer Interview Question Bank

[Databricks](#) · [PySpark](#) · [Delta](#) · [Streaming](#) · [Unity Catalog](#)

A structured bank of data engineering interview questions spanning Spark architecture and internals, PySpark, Delta Lake, streaming, Unity Catalog, cloud platforms and orchestration — organised so a topic can be revised on its own.

This document contains

- 22 sections covering the core Databricks data engineering surface
- 500+ questions, grouped by topic
- Spark internals, PySpark, Delta Lake, streaming, Unity Catalog, cloud and orchestration

Vivek Pandey — Data Engineer

Azure · Databricks · PySpark · Delta Lake

vp171097.github.io/DevWebsite

1. Spark Architecture & Internals

- Explain the Spark execution model: Driver → SparkContext → Cluster Manager → Executors → Tasks.
- What is a Job, Stage, and Task? What creates a new stage?
- Narrow vs wide transformations — give three examples of each.
- What is a shuffle? Why is it expensive? What happens on disk and over the network?
- Lazy evaluation — why does Spark do it, and what triggers execution?
- Difference between transformation and action. List 8 actions.
- What is a DAG? How does the DAG Scheduler differ from the Task Scheduler?
- RDD vs DataFrame vs Dataset. Why is DataFrame faster in PySpark?
- What is Catalyst Optimizer? Walk through its four phases (analysis → logical optimization → physical planning → code generation).
- What is Tungsten? What does whole-stage codegen do?
- What is Adaptive Query Execution (AQE)? Name its three main features (coalesce shuffle partitions, skew join handling, dynamic join strategy switching).
- Explain `explain()` output — how do you read a physical plan? What is `AdaptiveSparkPlan.isFinalPlan=false`?
- What is dynamic partition pruning?
- Executor memory layout: reserved / user memory / storage memory / execution memory. What is unified memory management?
- What is `spark.executor.cores` vs `spark.executor.instances` vs `spark.executor.memoryOverhead`?
- What causes an OOM on the driver vs on an executor? How do you fix each?
- What does `collect()` do and why is it dangerous?
- Difference between `cache()` and `persist()`. What storage levels exist?
- When does caching actually hurt performance?
- What is a broadcast variable? An accumulator?
- What is speculative execution?
- What is the default number of shuffle partitions and why is 200 often wrong?
- How does Spark decide the number of partitions when reading a file?
- What is a partition vs a bucket?

2. Performance Tuning & Troubleshooting

- Your job suddenly takes 3× longer with the same data volume. Walk through your investigation.
- How do you detect data skew from the Spark UI?
- Three ways to fix skew: AQE skew join, salting, broadcast join. Explain salting with code.
- How does broadcast join work? What is `spark.sql.autoBroadcastJoinThreshold` (default 10 MB)? When should you NOT broadcast?
- Join strategies: Broadcast Hash Join, Sort Merge Join, Shuffle Hash Join, Broadcast Nested Loop, Cartesian. When does Spark pick each?

- What is the small files problem? Why does it kill performance? How do you fix it?
- Target file size for Delta — what and why? (~128 MB–1 GB; `delta.targetFileSize`.)
- `repartition()` vs `coalesce()` — which shuffles, which doesn't, when do you use each?
- `repartition(col)` vs `partitionBy(col)` on write — completely different things, explain both.
- How do you avoid Python UDFs? What are `pandas_udf` / Arrow-optimized UDFs and why are they faster?
- Predicate pushdown and column pruning — which file formats support them?
- Parquet vs ORC vs Avro vs JSON vs CSV — when do you use each? Row vs columnar.
- What is the cost of `count()` on a Delta table vs a Parquet directory?
- Why is `df.count()` inside a loop an anti-pattern?
- Explain checkpointing (RDD/DataFrame `checkpoint()`) vs caching vs streaming checkpoint — three different things.
- How do you break long lineage chains?
- Spark UI tour: which tabs do you check first for a slow job? (Stages → longest task, SQL → plan, Executors → GC time, Storage.)
- High GC time — what does it mean and how do you fix it?
- Spill to disk — where do you see it and what causes it?
- How do you tune `spark.sql.files.maxPartitionBytes`?

3. Structured Streaming (deep)

Fundamentals

- Micro-batch vs continuous processing. What is the latency floor of each?
- How does Structured Streaming differ from DStreams?
- Explain the incremental execution model — how is a streaming query just an unbounded table?
- Output modes: Append, Update, Complete. Which aggregations require which?
- Trigger types: default, `processingTime`, `availableNow`, `once` (deprecated), `continuous`. When do you use `availableNow`?

Checkpointing (they will dig here)

- What is stored in a checkpoint directory? (`offsets/`, `commits/`, `sources/`, `state/`, `metadata/`)
- Explain the write-ahead log and how offsets guarantee exactly-once with a Delta sink.
- What happens if you delete the checkpoint? What if you point a new query at an existing checkpoint?
- Which query changes are checkpoint-compatible and which are not?
- Safe: adding filters, adding output columns, changing trigger interval, changing cluster size
- Unsafe: changing source, changing aggregation keys, changing output mode, changing watermark column, removing stateful ops
- Can two streaming queries share a checkpoint location? (No — one per query.)
- How do you recover from a corrupted checkpoint without reprocessing everything?
- What is `checkpointLocation` vs Delta's `_delta_log`?
- Does a checkpoint store the data itself? (No — only offsets and state.)

Watermarks & state

- What is a watermark and what problem does it solve?
- What happens to a record arriving after the watermark has passed?
- `withWatermark("ts", "10 minutes")` — explain exactly what the 10 minutes means.
- Watermark in stream-stream joins — why do you need one on BOTH sides?

- Inner vs left outer stream-stream join — why does outer join need a watermark to emit nulls?
- State store: HDFS-backed vs RocksDB. When do you switch to RocksDB?
- What is `flatMapGroupsWithState` / `transformWithState`? Give a use case (sessionization).
- `dropDuplicates` vs `dropDuplicatesWithinWatermark` — why does the first one grow state forever?
- How do you monitor state size? What is `StreamingQueryProgress`?
- What is state rebalancing / changelog checkpointing in RocksDB?

Delivery semantics & sinks

- Exactly-once, at-least-once, at-most-once — which does Spark give you and under what conditions?
- Why is a Delta sink exactly-once but a JDBC sink only at-least-once?
- `foreachBatch` — why is it needed for MERGE? How do you make it idempotent? (`txnAppId` + `txnVersion`, or dedupe on `batchId`.)
- Can you write to multiple sinks in one query? How?
- What is idempotent write in Delta (`.option("txnAppId", ...)`)?

Operations

- `maxFilesPerTrigger` / `maxBytesPerTrigger` — backpressure control.
- `StreamingQueryListener` — what would you monitor? (inputRowsPerSecond, processedRowsPerSecond, batchDuration, numRowsTotal in state.)
- What does it mean when `processedRowsPerSecond` < `inputRowsPerSecond`?
- How do you gracefully stop and restart a streaming job?
- How do you handle schema changes in a running stream?
- Streaming from a Delta table: `ignoreChanges`, `ignoreDeletes`, `skipChangeCommits`, `startingVersion`, `startingTimestamp` — explain each.
- Why does a MERGE into an upstream Delta table break a downstream stream, and how do you fix it?
- `maxOffsetsPerTrigger` for Kafka.
- How do you replay a stream from the beginning?
- Streaming aggregations: why can't you do arbitrary aggregations in Append mode?

4. Auto Loader (cloudFiles)

- What is Auto Loader and how is it different from `spark.readStream.format("json")` on a directory?
- Directory listing mode vs file notification mode — pros, cons, and cloud resources created (Event Grid + Queue on Azure; SNS + SQS on AWS).
- What is incremental listing? Why was it deprecated in favour of file events?
- How does Auto Loader track which files it has seen? (RocksDB in the checkpoint.)
- Schema inference: `cloudFiles.schemaLocation`, `inferColumnTypes`, schema hints.
- `schemaEvolutionMode` options: `addNewColumns`, `rescue`, `failOnNewColumns`, `none`. What happens to the stream on a new column with each?
- What is `_rescued_data` and why is it your best friend for messy JSON?
- How do you handle 200K small JSON files per day efficiently?
- `cloudFiles.maxFilesPerTrigger` and `cloudFiles.maxBytesPerTrigger`.
- Backfilling: `cloudFiles.backfillInterval`.
- `allowOverwrites` — what breaks if a file is replaced with the same name?
- How do you handle corrupt files? (`badRecordsPath`, `mode=PERMISSIVE/DROPMALFORMED/FAILFAST`.)
- Can Auto Loader read from multiple paths / a glob?
- Cost comparison: listing vs notification at 1M files/day.

5. Kafka / Event Hubs / Event-driven

- How do you read from Kafka in Spark? Required options (`kafka.bootstrap.servers` , `subscribe` , `startingOffsets`).
 - `startingOffsets` : `earliest` , `latest` , specific JSON offsets.
 - How does Spark commit Kafka offsets? Does it use Kafka consumer groups? (No — it manages offsets in the checkpoint.)
 - What happens if Kafka retention expires before you process the data?
 - `failOnDataLoss` — what does setting it to false actually mean?
 - Kafka partitions vs Spark partitions — what's the relationship?
 - How do you increase parallelism when reading Kafka?
 - Deserializing Avro with Schema Registry (`from_avro`).
 - Deserializing JSON: `from_json` with an explicit schema — why never infer in streaming?
 - Writing back to Kafka — key, value, topic columns.
 - At-least-once vs exactly-once with Kafka sink.
 - Event Hubs Kafka-compatible endpoint — how do you connect from Databricks?
 - Ordering guarantees — is global ordering possible?
-

6. Delta Lake

Core

- What is Delta Lake? How does it add ACID on top of Parquet?
- Explain `_delta_log` : JSON commit files, checkpoint Parquet files every 10 commits, `_last_checkpoint` .
- How does Delta achieve atomicity? (Optimistic concurrency + atomic rename/put-if-absent.)
- What is optimistic concurrency control? When do you get a `ConcurrentAppendException` and how do you avoid it?
- Which operations conflict with which? (Append vs Append is fine; Delete vs Delete on the same partition conflicts.)
- What is the transaction log protocol version / reader-writer version?
- Time travel: `VERSION AS OF` , `TIMESTAMP AS OF` . What limits how far back you can go?
- `DESCRIBE HISTORY` — what can you learn from it?
- `RESTORE TABLE ... TO VERSION AS OF n` .

Maintenance

- `OPTIMIZE` — what does it do? Bin-packing.
- `ZORDER BY` — how does it work (space-filling curve), how many columns max (3-4), when is it useless?
- **Liquid Clustering** — what problem does it solve vs partitioning + Z-order? `CLUSTER BY` , `CLUSTER BY AUTO` . Can you change clustering keys later? (Yes — that's the point.)
- `VACUUM` — what does it delete? Why the 7-day default? `RETAIN n HOURS` takes HOURS.
- `delta.deletedFileRetentionDuration` vs `delta.logRetentionDuration` — the difference matters.
- Why must you never VACUUM below the longest-running concurrent read?
- `spark.databricks.delta.retentionDurationCheck.enabled` — when and why disable it, and what's the risk?
- Auto Optimize: `optimizeWrite` + `autoCompact` . What do they cost you on write?
- What is Predictive Optimization?

Features

- MERGE syntax including `WHEN NOT MATCHED BY SOURCE`.
- How do you make MERGE fast? (Partition/cluster pruning in the ON clause, reduce source size, avoid full-table scan.)
- Why does MERGE fail with "multiple source rows matched"? How do you fix it?
- Change Data Feed (CDF): `delta.enableChangeDataFeed`, reading `table_changes()`, `_change_type` values.
- Deletion Vectors — what are they, why do they make DELETE/UPDATE fast, and what do they mean for downstream readers?
- Generated columns.
- Column mapping mode (`name` vs `id`) — required for rename/drop.
- Schema enforcement vs schema evolution. `mergeSchema`, `overwriteSchema`, `autoMerge`.
- Constraints: `NOT NULL`, `CHECK`.
- Identity columns.
- Shallow clone vs deep clone — use cases (test env, backup).
- `REPLACE WHERE` for dynamic partition overwrite.
- Delta UniForm (Iceberg/Hudi interop).
- Managed vs external Delta tables — what happens on DROP?

7. Medallion Architecture & Data Modeling

- Explain Bronze / Silver / Gold. What belongs in each?
- Should Bronze be schema-on-read? Do you dedupe in Bronze?
- How do you handle reprocessing from Bronze?
- Star schema vs snowflake schema. Fact and dimension tables.
- SCD Type 0/1/2/3/4/6 — explain each, then implement Type 2 in Delta MERGE.
- Surrogate keys in a distributed system — why is `monotonically_increasing_id()` risky? Alternatives (hash keys, identity columns).
- Fact table grain — why does defining it first matter?
- Additive, semi-additive, non-additive measures.
- How do you handle late-arriving dimensions?
- Data vault vs dimensional modeling — when would you pick data vault?
- Normalization vs denormalization in a lakehouse — why do we denormalize in Gold?
- Idempotency: how do you design a pipeline so a rerun produces the same result?
- Full load vs incremental load vs CDC. How do you decide?
- Watermark table pattern for incremental extraction.
- How do you handle hard deletes from a source system?
- Partitioning strategy: what makes a good partition column? Why is high cardinality bad? Why is date usually right?

8. Databricks Platform: Clusters, Jobs, Workflows

Clusters

- All-purpose vs Job cluster vs SQL Warehouse vs Serverless — cost, startup time, use case.
- Access modes: Dedicated (single user), Standard (shared), No-isolation. Which support UC?

- Databricks Runtime versions, LTS policy, ML runtime, Photon.
- What does Photon accelerate and what falls back to Spark? (Python UDFs, some RDD ops, certain data types.)
- Autoscaling — how does it work, and why is it risky for streaming?
- Instance pools — what problem do they solve?
- Spot / preemptible instances — when are they safe?
- Driver node sizing — when do you need a big driver?
- Single node cluster — when is it appropriate?
- Init scripts — cluster-scoped vs global. Where must they live under UC? (Volumes or workspace files; DBFS init scripts are deprecated.)
- Cluster policies — what are they for and what can they enforce?
- How do you size a cluster for X GB of data? Walk through the partition math.
- Serverless compute — what do you give up? (Cluster config control, some Spark configs, init scripts.)
- What are DBUs and how is cost calculated?

Jobs / Workflows

- Multi-task jobs, task dependencies, conditional (`if/else`) tasks, `for each` task.
- Job parameters and task values (`dbutils.jobs.taskValues`).
- Retry policies, timeouts, max concurrent runs.
- Task types: notebook, JAR, Python wheel, dbt, SQL, DLT pipeline, Run Job.
- How do you pass data between tasks?
- Job clusters shared across tasks vs per-task.
- Alerting and notifications; webhook destinations.
- How do you trigger a job on file arrival? (File arrival trigger.)
- Continuous jobs vs scheduled.
- Databricks Asset Bundles (DABs) — what problem do they solve?

Other platform

- Widgets (`dbutils.widgets`) for parameterization.
- `dbutils.notebook.run` vs `%run` — what's the difference?
- Secret scopes: Databricks-backed vs Azure Key Vault-backed. Never hardcode credentials.
- Repos / Git folders.
- DBFS vs Volumes vs external locations — DBFS root is legacy, don't put production data there.
- Databricks Connect.
- System tables for billing/audit/lineage.

8.1 Cluster Overload & Underload — Detect and Handle

Likely questions

- How do you know a cluster is under-provisioned vs over-provisioned?
- Where in the Spark UI do you look first?
- Your job is slow — is it a data problem, a code problem, or a cluster problem? How do you tell them apart?
- You doubled the cluster size and the job got no faster. Why?
- How do you right-size a cluster without pure trial and error?

A. Overload (under-provisioned) — symptoms

Spark UI → Executors tab

- GC Time >10-15% of Task Time → memory pressure. Clearest single overload signal.
- Failed tasks climbing; executors disappearing from the list mid-run.

- Storage Memory pinned at the cap while caching.

Spark UI → Stages tab

- `Spill (Memory)` / `Spill (Disk)` non-zero → execution memory exhausted, Spark writing shuffle/sort data to disk.
- High `Scheduler Delay` → tasks queued waiting for a free core.
- Task count $\gg$ available cores across many waves.
- Max task duration $\gg$ median → that's **skew**, not overload. Distinguish these.

Cluster metrics / event log

- CPU pinned at 100% for the entire run; memory flat at the ceiling.
- Event log: executor lost, `Container killed by YARN for exceeding memory limits`, exit code 137 (OS OOM kill), heartbeat timeouts.
- Driver OOM specifically → `collect()`, `toPandas()`, oversized broadcast, or too many concurrent tasks reporting back — a code issue, not worker sizing.

Streaming-specific

- `batchDuration` > trigger interval → permanently falling behind.
- `processedRowsPerSecond` < `inputRowsPerSecond`.
- Rising backlog: Kafka consumer lag, or Auto Loader unprocessed file count.
- `stateOperators.numRowsTotal` growing without bound → missing or too-loose watermark.

B. Overload — fixes, in the order you should try them

1. **Fix the code first.** Filter and project early, drop unused columns, kill `collect()`, replace Python UDFs with built-ins or `pandas_udf`, cache only what's genuinely reused.
2. **Fix partitioning.** Too few partitions = each task handles too much = spill/OOM. Target ~128 MB per partition; `spark.sql.shuffle.partitions` $\approx$ total executor cores (or a small multiple).
3. **Enable AQE** (default on in modern DBR) — coalesces partitions and handles skew joins automatically.
4. **Address skew** before scaling — salting, `broadcast()` the small side, AQE skew join. Bigger hardware never fixes skew; one task still does all the work.
5. **Reduce shuffle** — broadcast joins, pre-aggregate, filter before join.
6. **Then scale vertically** — memory-optimized instances for memory pressure; raise `spark.executor.memoryOverhead` for off-heap/Python overhead.
7. **Then scale horizontally** — more workers, or autoscaling with a sensible max.
8. **For streaming, apply backpressure instead of scaling:** `maxFilesPerTrigger`, `maxBytesPerTrigger`, `maxOffsetsPerTrigger`; switch the state store to RocksDB; tighten the watermark.
9. **Split the job** — one notebook doing five unrelated heavy things becomes several tasks with different cluster sizes.

C. Underload (over-provisioned) — symptoms

- **Idle executors** — cores allocated, task count near zero.
- **Fewer tasks than total cores.** Classic: 20 partitions on a 64-core cluster → 44 cores idle for the whole stage.
- **CPU utilization flat and low** (<30%) across the run.
- **Autoscaling never scales past the minimum** — max was set for a workload that doesn't exist.
- **Adding nodes doesn't reduce runtime** — bound by partition count, a single-threaded driver step, or an external system (JDBC source, API rate limit), not by compute.

- **Long idle tail** — cluster alive after the job finishes; auto-termination too generous, or it's an all-purpose cluster.
- **Cost per run high relative to data volume** — check `system.billing.usage` against the job's input bytes.

D. Underload — fixes

- **Right-size down.** 10 GB with simple transformations = single node or 2 small workers. Don't spin 8 workers for a 5-minute job.
- **Increase parallelism instead of nodes** when partition count is the bottleneck — `repartition()` to match core count.
- **Check the source.** JDBC read with no `partitionColumn / numPartitions` gives you exactly one task regardless of cluster size. Same for a single non-splittable gzip file.
- **Use job clusters, not all-purpose** — they terminate the moment the job ends.
- **Set auto-termination** aggressively (10–20 min) on all-purpose clusters.
- **Turn off autoscaling** for small predictable jobs — scale-up latency costs more than it saves.
- **Consider serverless** for short bursty jobs to eliminate idle and startup cost.
- **Use instance pools** if startup latency is why people leave clusters running.
- **Enforce with cluster policies** — cap max workers and node types.

E. Right-sizing method (say this out loud in the interview)

1. Input size ÷ 128 MB = target partition count. *(10 GB ≈ 80 partitions.)*
2. Total cores ≈ partition count ÷ 2–4, so each core handles a few tasks.
3. Memory per core ≈ 4–8 GB for typical ETL; more if caching or doing wide aggregations.
4. Run once, then read the Spark UI: **spill → add memory; idle cores → remove nodes; skew → fix the code.**
5. Iterate once or twice, then pin the config in the job definition.

F. Distinctions they may probe

Symptom	It's actually
One task takes 10× the others	Skew — not overload
Spill to disk + high GC	Genuine memory overload
High scheduler delay, all cores busy	CPU overload — add cores
20 tasks on 64 cores	Underload — repartition, don't add nodes
Adding workers changes nothing	Bottleneck elsewhere (source, driver, partition count)
Driver OOM only	Code problem (<code>collect</code> , broadcast), not worker size
Streaming batch time creeping up	State growth or backlog — check watermark before scaling

8.2 Clusters — Detailed Reference

A. Compute types

Type	Lifecycle	Cost	Use for
All-purpose (interactive)	Manual start/stop, auto-terminate	Highest DBU rate	Development, ad-hoc analysis, notebooks
Job cluster	Created at job start, destroyed at end	~½ the DBU rate of all-purpose	All scheduled production work
SQL Warehouse (classic/pro)	Auto start/stop on query	SQL DBU rate	BI tools, dashboards, SQL analysts
Serverless SQL Warehouse	Instant start (seconds)	Highest per-second, lowest idle waste	Bursty BI, short queries
Serverless jobs/notebooks	No cluster to manage	Per-second billing	Short jobs, teams who shouldn't tune Spark
Instance pool	Pre-warmed idle VMs	Pay cloud VM cost while idle, no DBU	Cutting 5-min startup to ~30s

Interview point: using an all-purpose cluster for a scheduled job is the single most common cost mistake — you pay roughly double the DBU rate *and* the cluster often idles between runs.

B. Access modes (this is where UC compatibility lives)

Mode	Users	UC support	Languages	Notes
Dedicated (was Single User)	One user or one group	Full	Python, SQL, Scala, R	Needed for ML workloads, Scala, streaming with some libs
Standard (was Shared)	Multiple concurrent	Full	Python, SQL, Scala (DBR 13.3+)	Process isolation per user; some Spark APIs restricted
No isolation shared	Multiple	None	All	Legacy. No UC, no credential passthrough. Avoid.

Restrictions on Standard mode worth knowing: no DBFS mounts for UC data, no `spark.sparkContext` / RDD APIs, no arbitrary JVM access, limited `dbutils.fs` on non-UC paths, init scripts must be on Volumes or workspace files (allowlisted).

C. Databricks Runtime (DBR)

- **Versioning:** major.minor, e.g. 14.3, 15.4, 16.x. LTS releases (13.3, 14.3, 15.4) get ~3 years support — **pin production to LTS**.
- **Flavours:** standard, **ML** (adds PyTorch/TensorFlow/MLflow/scikit, GPU variants), **Photon-enabled**, Light (deprecated).
- Each DBR bundles a specific Spark version, Delta version, Python version, and preinstalled library set. Upgrading DBR can silently change library versions — regression-test before bumping.
- **Photon:** vectorized C++ query engine. Accelerates SQL/DataFrame ops, joins, aggregations, Delta writes. Falls back to Spark for Python/Scala UDFs, RDD ops, some complex types, streaming state ops. Costs ~2× DBU/hour but often finishes in <½ the time — evaluate on *total job cost*, not hourly rate.

D. Nodes, Workers, Executors, Cores, Tasks

The hierarchy — be able to draw this on a whiteboard

Cluster

- └─ Driver node (1) → JVM running SparkContext + your Python process
- └─ Worker nodes (N) → each runs 1 Executor JVM
 - └─ Executor has C cores (slots)
 - └─ Each core runs 1 Task at a time
 - └─ 1 Task = 1 Partition of data

Key relationships to state precisely:

- **1 worker node = 1 executor** on Databricks (unlike open-source Spark/YARN, where you can pack several executors per node).
- **Executor cores = VM vCPUs**. A 16-vCPU worker gives 16 task slots.
- **Total parallelism = number of workers × cores per worker**. 8 workers × 8 cores = 64 concurrent tasks.
- **Tasks ≠ cores**. If a stage has 200 partitions on 64 cores, Spark runs them in ~4 waves.
- Autoscaling changes N (workers), never C (cores per worker) — cores per worker is fixed by the VM type you pick.

Driver node

- Runs SparkContext, the DAG scheduler, the task scheduler, and the notebook's Python interpreter.
- Collects results from executors and hosts broadcast variables before distribution.
- Runs any non-Spark code you write — pandas, requests, loops.
- **Size it up when:** thousands of partitions (scheduling overhead), large broadcasts, `collect()` / `toPandas()`, several concurrent streaming queries on one cluster, heavy single-node Python.
- **Rule of thumb:** driver ≈ same size as a worker for normal ETL; 2-4× a worker if you're doing driver-heavy work.
- Driver failure kills the whole cluster; a worker failure only loses that executor's tasks (Spark re-runs them).
- **Driver OOM symptoms:** notebook dies, `Driver is up but not responsive`, `java.lang.OutOfMemoryError` in the *driver* log, cluster auto-restarts. This is nearly always a code problem, not a sizing problem.

Worker nodes

- Execute tasks, hold cached data in Storage Memory, and write shuffle files to local disk.
- More workers = more parallelism; bigger workers = more memory per task.
- **Fewer big workers vs more small workers:** fewer/bigger reduces shuffle network traffic and helps broadcast joins and caching; more/smaller gives finer autoscaling granularity and better spot-loss tolerance. For most ETL, mid-size nodes (8-16 vCPU) are the sweet spot — avoid both 4-vCPU sprawl and 64-vCPU monoliths.

Executor memory breakdown (this is a favourite question)

For a worker with, say, 64 GB:

```

Total JVM heap (spark.executor.memory)
├─ Reserved memory (~300 MB)
└─ Usable = (heap - 300MB)
    ├─ Unified Memory (spark.memory.fraction, default 0.6)
    │   ├─ Execution memory → shuffles, joins, sorts, aggregations
    │   └─ Storage memory → cached/persisted DataFrames, broadcasts
    │       (they borrow from each other; execution can evict storage,
    │       but storage cannot evict execution)
    └─ User memory (0.4) → your data structures, UDF objects
Plus, OUTSIDE the heap:
└─ spark.executor.memoryOverhead → Python workers (PySpark!), off-heap,
    network buffers. Default max(384MB, 10% of executor memory).

```

- **PySpark runs Python in separate processes outside the JVM heap** — that memory comes from overhead. Heavy `pandas_udf` / Python UDF work is the #1 reason to raise `memoryOverhead`.
- `Container killed by YARN for exceeding memory limits` / exit 137 = overhead exhausted, not heap.

Single node cluster

- Driver only, `spark.master = local[*]`. Executors run inside the driver JVM.
- Good for: <a few GB, library testing, pandas/scikit work, cheap dev.
- Bad for: anything needing real shuffle parallelism or resilience.

D2. Instance families and node types

Family	Azure	AWS	vCPU:RAM	Use when
General purpose	D-series (<code>Standard_D4ds_v5</code> , <code>D8ds_v5</code>)	<code>m5d.xlarge</code> , <code>m6gd</code>	1:4	Default ETL, balanced read/transform/write
Memory optimized	E-series (<code>Standard_E8ds_v5</code> , <code>E16ds_v5</code>)	<code>r5d</code> , <code>r6gd</code> , <code>x2gd</code>	1:8	Wide aggregations, big joins, heavy caching, fixing spill
Compute optimized	F-series (<code>Standard_F8s_v2</code>)	<code>c5d</code> , <code>c6gd</code>	1:2	CPU-bound parsing/regex/JSON, Photon SQL, compression
Storage optimized	L-series (<code>Standard_L8s_v3</code>)	<code>i3.xlarge</code> , <code>i4i</code>	1:8 + huge NVMe	Shuffle-heavy jobs, Delta/disk cache, repeated scans
GPU	NC/ND series	<code>p3</code> , <code>g4dn</code> , <code>g5</code>	—	Deep learning only. Never for ETL.

Practical rules

- **Always pick a node type with local SSD/NVMe** (the `d` in `D4ds_v5` , `m5d` , `c5d`). Shuffle spill and the Delta disk cache go to local disk — remote-disk-only VMs are dramatically slower.
- **Start general purpose.** Move to memory-optimized only after you see spill/GC in the Spark UI; move to compute-optimized only if CPU is pinned with no spill.
- **Graviton (AWS `m6gd` / `r6gd`) / ARM** gives ~20-40% better price-performance — but check library compatibility.
- Match driver and worker family unless you have a specific reason not to.
- Don't mix wildly different sizes across a cluster; Spark assumes homogeneous executors.

Sizing worked example — 10 GB, simple transformations

$10\text{ GB} \div 128\text{ MB} \approx 80\text{ partitions}$
 $80\text{ partitions} \div 2\text{ waves} \approx 40\text{ cores...}$ but this is a SIMPLE job,
 so oversizing wastes money.
 Practical answer: $2\text{ workers} \times 8\text{ vCPU} = 16\text{ cores}$ (~5 waves), or
 1 single-node 16-vCPU cluster.
 Memory: 4-8 GB per core → D8ds_v5 (8 vCPU / 32 GB) fits fine.
 Set `spark.sql.shuffle.partitions` $\approx 16-64$ (NOT the default 200).
 Job cluster, Photon on if it's SQL-shaped, no autoscaling.

Sizing worked example — 2 TB daily batch with joins

$2\text{ TB} \div 128\text{ MB} \approx 16,000\text{ partitions}$
 Target 3-4 waves → ~4,000-5,000 cores is too many; instead
 accept more waves: 128-256 cores.
 $128\text{ cores} \div 16\text{ vCPU} = 8\text{ workers}$ (E16ds_v5, memory-optimized
 because of the joins)
 Driver: same size or one step up.
`spark.sql.shuffle.partitions` $\approx 2,000-4,000$ (multiple of core count).
 Enable AQE, Photon, autoscale 4→12.

Prefer instance types with **local NVMe/SSD** for shuffle-heavy jobs — shuffle spill goes to local disk, and remote-disk-only VMs are dramatically slower.

E. Autoscaling

- **How it works:** Databricks adds workers when tasks are pending, removes them when idle for a period, respecting min/max workers. Scale-up is aggressive, scale-down conservative (it won't remove an executor holding shuffle data still needed).
- **Optimized autoscaling** (jobs) can scale down mid-job; standard autoscaling on all-purpose is slower to release.
- **Why it's risky for streaming:** continuous micro-batches keep executors busy, so it rarely scales down; and scale-down mid-stream can force state reshuffling and lengthen batches. Prefer a fixed size for streaming.
- **When it helps:** variable-volume batch loads, backfills, jobs where stage sizes differ wildly.
- **When it hurts:** short jobs (VM acquisition takes minutes — the job ends before help arrives), and small predictable jobs where it just adds latency.
- Set `min_workers` to what the *steady state* needs, not 1, so you don't pay a scale-up penalty every run.

F. Spot / preemptible instances

- Cheap (up to ~70-90% off) but can be reclaimed by the cloud provider at any time.
- **Safe for:** batch jobs with retries, dev clusters, stateless work.
- **Risky for:** long-running streaming (state loss + recovery), jobs with tight SLAs, anything where a lost executor means recomputing hours of lineage.
- **Best practice:** on-demand driver + spot workers, with a `first_on_demand` count so the first N workers are stable.
- Set a max spot price / fallback-to-on-demand policy so jobs don't hang waiting for capacity.

G. Configuration you should be able to name

Spark configs

```
spark.sql.shuffle.partitions          # default 200 – tune to core count
spark.sql.adaptive.enabled            # AQE, default true on modern DBR
spark.sql.adaptive.coalescePartitions.enabled
spark.sql.adaptive.skewJoin.enabled
spark.sql.autoBroadcastJoinThreshold # default 10MB; -1 disables
spark.sql.files.maxPartitionBytes     # default 128MB read partition size
spark.executor.memoryOverhead        # off-heap; raise for Python/pandas UDFs
spark.databricks.delta.optimizeWrite.enabled
spark.databricks.delta.autoCompact.enabled
spark.databricks.io.cache.enabled     # Delta cache (disk caching of Parquet)
```

Environment variables & secrets

- Never hardcode credentials in a cluster config. Use `{{secrets/scope/key}}` syntax in Spark conf or env vars, backed by a Databricks or Key Vault secret scope.

Libraries

- Cluster-scoped (installed on cluster start), notebook-scoped (`%pip install`), or workspace/UC-scoped.
- Prefer notebook-scoped `%pip` for dev, and a **wheel + Databricks Asset Bundle** for production — reproducible and version-pinned.
- Cluster-scoped library installs add to startup time and can fail the cluster.

Init scripts

- Cluster-scoped or global. Under UC they must live in **Volumes or workspace files** and be allowlisted; **DBFS init scripts are deprecated/disabled**.
- A failing init script fails the whole cluster launch — check the cluster event log and init script logs.

Tags

- Custom tags flow into `system.billing.usage` and into cloud provider billing. This is how you attribute cost per project/team/client. Expect the question "how do you track cost per pipeline?" — the answer is tags + system tables.

H. Cluster policies

- JSON rule sets that constrain what users can create: allowed node types, max workers, max DBU/hour, mandatory auto-termination, mandatory tags, fixed DBR version, forced access mode.
- Policy types: `fixed`, `allowlist`, `blocklist`, `range`, `regex`, `unlimited`, plus `defaultValue` / `hidden`.
- Real-world use: a "Job — Small ETL" policy capping at 4 workers with auto-terminate 20 min, so nobody spins a 20-node cluster to read a lookup table.
- Databricks ships default families (Personal Compute, Shared Compute, Job Compute).

I. Lifecycle, startup, and logs

- **Startup phases**: request VMs from cloud → boot → install DBR image → run init scripts → install libraries → register with control plane → RUNNING. Typically 3–7 minutes; pools cut this to ~30s.
- **Auto-termination**: minutes of inactivity before shutdown. Set 10–30 min on all-purpose; irrelevant for job clusters.
- **Cluster event log** (UI → cluster → Event log): CREATING, STARTING, RUNNING, RESIZING, TERMINATING, plus reasons — this is where you find "spot instance reclaimed", "init script failed", "cloud provider capacity".

- **Driver logs** (stdout/stderr/log4j): where Python tracebacks and OOM stack traces land.
- **Log delivery**: configure cluster logs to persist to a Volume/S3/ADLS, otherwise logs vanish with the cluster — critical for debugging job clusters after the fact.
- **Metrics**: modern Databricks uses the built-in compute metrics UI (CPU, memory, network, disk); Ganglia on older DBR.

J. Cost mechanics

- **Total cost = DBU rate × time + underlying cloud VM cost.** People forget the second half.
- DBU rate varies by workload type (jobs < all-purpose < SQL), tier (Standard/Premium/Enterprise), and Photon.
- Levers: job clusters over all-purpose, auto-termination, spot workers, pools, right-sizing, Photon where it wins, serverless for bursty work, and killing idle dev clusters.
- Query `system.billing.usage` joined to `system.compute.clusters` and your tags to build a cost-per-pipeline report.

K. Cluster troubleshooting cheat sheet

Problem	Likely cause	Fix
Cluster won't start	Cloud quota, subnet IP exhaustion, init script failure, unavailable instance type	Check event log; raise quota; change node type
Very slow start	Library installs, init scripts, no pool	Use pools; move libs to a wheel
Executors keep dying	Spot reclamation, OOM, GC thrash	On-demand workers; more memory; fix partitions
Container killed... exceeding memory limits	Off-heap/Python overhead	Raise <code>spark.executor.memoryOverhead</code>
Driver not responsive	<code>collect()</code> / <code>toPandas()</code> , huge broadcast	Fix the code; bigger driver only as a stopgap
Job runs fine solo, fails when concurrent	Shared all-purpose cluster contention	Move to job clusters, one per job
Works on Dedicated, fails on Standard	RDD API / <code>sparkContext</code> / mount usage	Rewrite with DataFrame API + UC paths
Streaming query stops after cluster resize	Autoscaling churn	Fixed cluster size for streaming
"No space left on device"	Shuffle spill filling local disk	Bigger local-disk instances; reduce shuffle

L. Questions they might ask, with the shape of a good answer

- "Walk me through choosing a cluster for a new pipeline." → Volume → partition math → cores → memory per core → job cluster → Photon if SQL-heavy → LTS DBR → tags + policy → run once and read the Spark UI.
- "All-purpose or job cluster?" → Job cluster for anything scheduled; all-purpose only for interactive dev.
- "Why is your streaming cluster fixed size?" → Autoscaling churn reshuffles state and lengthens batches; backpressure options control load better.
- "How do you stop developers from overspending?" → Cluster policies + mandatory tags + auto-

termination + cost reporting from system tables.

- "How do you pick DBR?" → LTS for prod, match across dev/UAT/prod, regression-test before upgrading, ML runtime only where needed.

9. Unity Catalog & Governance

- Three-level namespace. How does it change from Hive metastore two-level?
- Metastore → catalog → schema → table / view / volume / function / model.
- One metastore per region — what does that mean for a multi-region client?
- Storage credential vs external location — explain the split of concerns.
- Managed vs external table under UC — data lifecycle on DROP.
- Managed vs external **volumes** — when do you use each?
- Migrating from Hive metastore: `SYNC SCHEMA/TABLE`, UCX toolkit, upgrade wizard.
- Why do `/mnt/` mounts not work with UC standard clusters? What replaces them?
- Privileges: `USE CATALOG`, `USE SCHEMA`, `SELECT`, `MODIFY`, `CREATE TABLE`, `READ VOLUME`, `WRITE VOLUME`, `EXECUTE`, `ALL PRIVILEGES`.
- Ownership vs grants. What can an owner do that a grantee can't?
- Inheritance model — grant at catalog level flows down.
- Dynamic views: `current_user()`, `is_account_group_member()` — how do you do row-level filtering with a view?
- Row filters and column masks (the newer native feature) — syntax and use case (mask PII in call transcripts, filter invoices by region).
- Attribute-Based Access Control / tags.
- Lineage — table and column level. What triggers lineage capture?
- Audit logs and `system.access.audit`.
- Delta Sharing — open sharing vs Databricks-to-Databricks. How do you share data with a client who isn't on Databricks?
- Catalog binding (workspace-catalog isolation).
- Identity federation: account-level users/groups/service principals; SCIM provisioning.
- Service principals for jobs — why not personal tokens?
- Lakehouse Federation — querying external DBs (Snowflake, SQL Server, Postgres) through UC.
- What is the UC metastore admin role vs account admin vs workspace admin?

10. Delta Live Tables / Lakeflow Declarative Pipelines

- What is DLT and what does it manage for you?
- Streaming table vs materialized view vs view in DLT.
- `@dlt.table` vs `@dlt.view` vs `create_streaming_table`.
- Expectations: `expect`, `expect_or_drop`, `expect_or_fail`. How do you track violations?
- `APPLY CHANGES INTO` — SCD Type 1 and Type 2 without writing MERGE.
- Development vs Production mode.
- Triggered vs Continuous pipeline mode.
- Full refresh vs incremental refresh — when does a full refresh happen automatically?

- Auto Loader inside DLT.
- Event log — what's in it and how do you query it?
- DLT vs hand-written Workflows — when would you NOT use DLT?

11. SQL Round (be ready to write)

- Second/Nth highest salary — three different ways.
- `RANK` vs `DENSE_RANK` vs `ROW_NUMBER` — with a worked example showing the difference.
- Running total, moving average (3-month), cumulative distinct count.
- `LAG` / `LEAD` for month-over-month change and change detection.
- Window frame: `ROWS BETWEEN` vs `RANGE BETWEEN`, `UNBOUNDED PRECEDING`, `CURRENT ROW`. What's the default frame with and without `ORDER BY`?
- `PARTITION BY` with aggregate vs `GROUP BY` — when you need all columns retained (your district/population question).
- Find and delete duplicates keeping one row.
- Gaps and islands (consecutive login streaks).
- Self join — employee/manager hierarchy.
- Recursive CTE for an org hierarchy.
- All join types including `ANTI` and `SEMI` joins; `EXISTS` vs `IN` vs `LEFT JOIN ... IS NULL`.
- `UNION` vs `UNION ALL` — performance difference.
- `HAVING` vs `WHERE` vs `QUALIFY` (Databricks supports `QUALIFY` — know it).
- `PIVOT` / `UNPIVOT`.
- `COALESCE`, `NULLIF`, `NVL`, and `NULL` behavior in aggregates, joins, and `NOT IN`.
- Date functions: `date_trunc`, `add_months`, `datediff`, `months_between`, `last_day`.
- `EXPLODE` / `LATERAL VIEW` for arrays; `POSEXPLODE`.
- Handling JSON in SQL: `get_json_object`, `from_json`, `:` colon notation, `variant` type.
- `GROUPING SETS`, `ROLLUP`, `CUBE`.
- Correlated vs uncorrelated subquery.
- CTE vs subquery vs temp view — performance implications.
- Write a query to find customers who bought in every month of 2024.
- Write a query to find the top 3 products per category.

12. PySpark Coding Round

Be ready to write these cleanly, from memory:

- Read a CSV/JSON with an explicit `StructType` schema (know `StructField`, `ArrayType`, `MapType`).
- Flatten a nested JSON with arrays and structs (recursive flatten function is a classic ask).
- `explode` vs `explode_outer` vs `posexplode`.
- `withColumn`, `withColumns`, `selectExpr`, `expr`.
- `when` / `otherwise` chains.
- Window function: `Window.partitionBy().orderBy().rowsBetween()`.
- Dedupe keeping latest record per key using `row_number()`.
- SCD Type 2 with `DeltaTable.merge()`.

- Broadcast join with `broadcast(df)`.
- `groupBy().agg()` with multiple aggregates and aliases.
- `pivot()`.
- `union` vs `unionByName(allowMissingColumns=True)`.
- Handling nulls: `fillna`, `dropna`, `na.replace`.
- `cast()` and type conversion errors.
- Reading with `mode="PERMISSIVE"` + `_corrupt_record`.
- Writing: `mode` (append/overwrite/error/ignore), `partitionBy`, `replaceWhere`.
- `pandas_udf` for vectorized operations.
- `foreachBatch` function signature: `def upsert(batch_df, batch_id):`
- Reading/writing to a UC table: `spark.table("cat.sch.tbl")`, `.saveAsTable()`.
- Convert a Spark DF to Pandas safely (`toPandas()` risk, `mapInPandas`).
- Unit-testable code: why you should write functions that take and return DataFrames.

13. Python Fundamentals (often a quick screen)

- List vs tuple vs set vs dict; mutability.
- List/dict comprehensions.
- `*args` / `**kwargs`.
- Decorators — write a simple timing decorator.
- Generators and `yield` — why do they matter for large data?
- Context managers / `with`.
- Exception handling, custom exceptions, `finally`.
- OOP: class, inheritance, `__init__`, `@staticmethod` vs `@classmethod`.
- Shallow vs deep copy.
- Lambda, `map`, `filter`, `reduce`.
- `is` vs `==`.
- GIL — what is it and why does it matter (or not) in Spark?
- Reading/writing JSON, working with `datetime`.
- Logging instead of `print`.
- Virtual environments, `requirements.txt`, wheels.

14. Cloud (Azure-first, AWS as backup)

Azure

- ADLS Gen2 vs Blob Storage — hierarchical namespace.
- Access from Databricks: service principal + OAuth, managed identity, access connector for UC.
- What is a storage account / container / directory hierarchy?
- Azure Key Vault-backed secret scope setup.
- ADF: pipelines, activities, linked services, datasets, integration runtimes (Azure IR vs Self-hosted IR).
- ADF Copy Activity vs Databricks notebook activity — when do you use each?
- ADF triggers: schedule, tumbling window, event-based. Difference between schedule and tumbling window.
- Azure Event Hubs vs Kafka.

- Synapse vs Databricks — how do you position them?
- Azure Monitor / Log Analytics for pipeline observability.
- Private endpoints, VNet injection, NCC — network security for Databricks.

AWS (if they're AWS)

- S3 storage classes, lifecycle policies, eventual consistency (now strong).
- IAM roles, instance profiles, assume-role for UC storage credentials.
- Glue Catalog vs Unity Catalog.
- Kinesis vs Kafka (MSK).
- Step Functions / MWAA for orchestration.
- Redshift vs Databricks SQL.

15. Orchestration & CI/CD

- Airflow: DAG, operator, task, sensor, XCom, backfill, catchup, `depends_on_past`.
- Airflow `DatabricksSubmitRunOperator` / `DatabricksRunNowOperator`.
- Idempotent DAG design; why `catchup=True` can flood you.
- Retries, SLAs, alerting.
- Git branching strategy for data pipelines (feature → dev → main).
- CI/CD for Databricks: Databricks Asset Bundles, `databricks` CLI, Azure DevOps / GitHub Actions pipeline stages.
- How do you promote code from dev → UAT → prod? How do you handle environment-specific catalog names?
- Testing: unit tests with `pytest` + local `SparkSession`, `chispa` for DataFrame assertions, integration tests on a small dataset.
- How do you test a streaming job?
- Code review standards for notebooks — why are notebooks hard to review, and what do you do about it? (Move logic to `.py` modules / wheels.)
- Infrastructure as code: Terraform Databricks provider.
- Secrets management in CI.
- Blue/green or canary deploy for a pipeline?

16. Data Quality, Observability, Reliability

- What checks do you run at each Medallion layer?
- Great Expectations / DLT expectations / Delta constraints — compare.
- How do you quarantine bad records instead of failing the job?
- Reconciliation: how do you prove Silver row counts match Bronze?
- How do you detect schema drift before it breaks production?
- Duplicate detection strategy at scale.
- Freshness SLAs — how do you monitor and alert?
- Lakehouse Monitoring in Databricks.
- What's your on-call runbook for a failed pipeline?
- How do you do root cause analysis on a data discrepancy reported by a business user?
- Backfill strategy: how do you backfill 2 years without disrupting daily loads?

- Disaster recovery: RPO/RTO for a lakehouse. Deep clone for backup.
- How do you handle a GDPR "right to be forgotten" request in Delta? (DELETE + VACUUM below retention + CDF implications.)

17. Cost Optimization

- Monthly cost doubled. Where do you look first?
- All-purpose vs job compute cost multiple.
- Auto-termination settings.
- Spot instances with on-demand driver.
- Photon: costs more DBUs per hour but often cheaper per job — how do you evaluate?
- Right-sizing: over-provisioned clusters idling.
- Storage cost: VACUUM not running, **RETAIN** set too high, orphaned files.
- Reducing shuffle to cut runtime.
- Serverless SQL warehouse auto-stop.
- Caching Gold tables vs recomputing.
- System billing tables — how do you attribute cost per job/team? (Tags!)
- Cluster policies to enforce cost guardrails.

18. Scenario / System Design Questions

1. Design an end-to-end pipeline: 200K JSON files/day from an API landing in ADLS → curated analytics tables. Walk through ingestion, layers, schema handling, orchestration, monitoring, cost.
2. Streaming job was down 8 hours; 1.5M files backed up. Catch up without OOM or blowing the SLA.
3. Duplicates appeared downstream after a job retry. Diagnose and fix permanently.
4. Client wants latency reduced from hourly to under 1 minute. What changes architecturally, and what's the cost delta?
5. Bad upstream load corrupted Bronze 3 days ago and it's propagated to Gold. Recovery plan.
6. A join is skewed — one country holds 80% of rows. Three fixes, with trade-offs.
7. Source adds a new nested field mid-week. Does your pipeline break? Design so it doesn't.
8. Client wants to keep only 5 years of data and stay audit-compliant. Design the retention job.
9. You need to share curated data with an external client who uses Snowflake. Options?
10. Two teams need the same Silver table but with different PII visibility. Design it.
11. Migrate 400 Hive metastore tables and 30 mount points to Unity Catalog with zero downtime.
12. Design a CDC pipeline from an on-prem SQL Server to the lakehouse.
13. A Gold dashboard query takes 45 seconds. Make it 2 seconds.
14. How would you build a data pipeline that must be reproducible from any point in the last 90 days?
15. Batch job and streaming job both write to the same Delta table. What goes wrong and how do you prevent it?
16. Design deduplication for events that may arrive up to 24 hours late.
17. You have to process files that arrive out of order with the same business key. How do you guarantee the latest wins?
18. Design monitoring/alerting for 40 pipelines so the on-call engineer isn't drowning.

19. Project Deep-Dive (they WILL drill your resume)

Have crisp numbers ready:

- **Adidas invoice pipeline**: source systems, volume/day, Bronze→Silver→Gold logic, table count, SLA, cluster config, runtime, cost, biggest failure you handled.
- **Amazon Connect transcripts (~200K JSON/day)**: file sizes, Auto Loader config, schema handling, nested structure flattening, how you solved small files, downstream consumers.
- **TotalEnergies streaming (Kafka + Azure Databricks)**: throughput, latency achieved, checkpoint strategy, state size, what broke in production and how you fixed it.
- **Team lead of 6 engineers**: how you split work, reviewed code, onboarded people, handled a missed deadline.

For each, be able to answer: *What was the hardest technical problem? What would you do differently now? What did you own vs the team?*

20. Behavioral

- Tell me about a production incident you caused. What happened and what changed after?
 - Describe a time you disagreed with an architecture decision.
 - How do you handle a client asking for something technically unwise?
 - Tight deadline vs doing it right — walk me through a real trade-off you made.
 - How do you keep up with Databricks changes?
 - Mentoring juniors — concrete example.
 - A stakeholder says the numbers are wrong. Your process?
-

21. Common Traps (know the precise answer)

Trap	Correct answer
<code>VACUUM RETAIN 5 YEARS</code> = data retention	No — it controls file/time-travel history, not business rows. Use DELETE for rows.
<code>RETAIN</code> unit	HOURS, not days. 5 years = 43800.
Reusing a checkpoint after logic change	Only safe for non-stateful changes.
<code>coalesce</code> always cheaper	It avoids shuffle but can under-parallelize upstream stages.
<code>cache()</code> always helps	Costs memory; useless for single-use DataFrames.
Broadcast anything small	Driver must collect it first — big broadcast = driver OOM.
More partitions = faster	Too many = scheduling overhead + small files.
Mounts work with UC	Not on standard-access clusters; use external locations/volumes.
DROP on external table deletes data	No — only metadata. Managed tables delete data.
<code>dropDuplicates</code> in streaming is free	Unbounded state growth without a watermark.
Z-order and Liquid Clustering together	Mutually exclusive — Liquid replaces partitioning + Z-order.
Auto Loader = streaming only	Works with <code>availableNow</code> for batch-style incremental.
<code>count(*)</code> on Delta scans data	It reads the transaction log stats — it's cheap.

22. Questions to Ask Them

- What does the current architecture look like — mostly batch, streaming, or both?
- Are you fully on Unity Catalog or still migrating from Hive metastore?
- What are the biggest reliability or cost pain points today?
- DLT/Lakeflow or hand-rolled Workflows? Why?
- How is the team structured and what would my first 90 days look like?
- What does the on-call rotation look like?
- What's the CI/CD maturity for data pipelines?

Night-Before Checklist

- [] Rehearse the 4 project stories out loud with numbers
- [] Write SCD2 MERGE from memory once
- [] Write a Kafka→dedupe→foreachBatch MERGE from memory once
- [] Write a nested-JSON flatten from memory once
- [] Say the checkpoint contents out loud
- [] Say the UC hierarchy + storage credential/external location split out loud
- [] Re-read the Traps table last thing